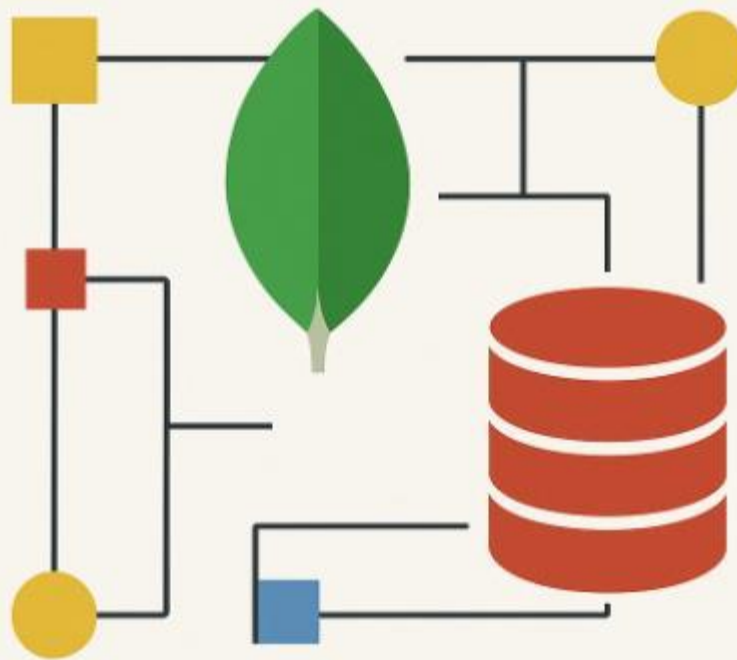


MongoDB

para informáticos que vienen de SQL Server



GUÍA

Introducción a la guía

En el mundo actual del desarrollo de software y gestión de datos, la flexibilidad y la adaptabilidad se han convertido en cualidades clave para cualquier profesional de TI. Esta guía nace precisamente con ese propósito: facilitar una transición rápida, clara y eficaz desde SQL Server a MongoDB, dos tecnologías que, aunque distintas en filosofía y estructura, pueden convivir y complementarse en múltiples escenarios.

Muchos informáticos con experiencia en SQL Server se enfrentan al reto de abordar modelos NoSQL como MongoDB, y es común sentir cierta frustración ante la falta de esquemas, joins tradicionales o procedimientos almacenados. Esta guía no pretende ser un manual exhaustivo de MongoDB, sino una herramienta de traducción mental y práctica, pensada específicamente para quienes ya dominan entornos relacionales.

¿Por qué esta guía es diferente?

- Compara conceptos SQL con su equivalente MongoDB directamente.
- Usa ejemplos reales y prácticos, enfocados en tareas cotidianas.
- Evita tecnicismos innecesarios y va al grano.
- Está estructurada de forma progresiva, siguiendo el camino lógico de quien ya ha trabajado con bases de datos.

Esta guía te acompañará paso a paso para que no desaprendas, sino que reinterpretes lo que ya sabes, aplicándolo con eficacia en un nuevo paradigma. Porque no se trata de olvidar SQL Server, sino de sumar MongoDB como una herramienta más a tu repertorio profesional.

Autor

Óscar de la Cuesta Campillo

 palentino.es

 [@oscardelacuesta](https://twitter.com/oscardelacuesta)



Licencia de uso

Este libro se distribuye bajo la siguiente licencia:

Creative Commons Attribution 4.0 International (CC BY 4.0)

Qué puedes hacer con esta obra

- ✓ **Compartir** — copiar y redistribuir el material en cualquier medio o formato
- ✓ **Adaptar** — remezclar, transformar y construir a partir del contenido, incluso para fines comerciales

Condiciones

- Debes dar crédito al autor original (Óscar de la Cuesta Campillo).
- Incluir un enlace a la licencia.
- Indicar si realizaste cambios.

Ejemplo de atribución:

“MongoDB para Informáticos de SQL Server: Guía de Transición y Dominio de Óscar de la Cuesta Campillo – palentino.es – @oscardelacuesta”



■ ÍNDICE

1. Introducción

- 1.1. Objetivo de la guía
- 1.2. ¿Por qué MongoDB?
- 1.3. Diferencias clave entre SQL Server y MongoDB

2. Fundamentos de MongoDB

- 2.1. Arquitectura general
- 2.2. Instalación y configuración
- 2.3. MongoDB Compass y la shell
- 2.4. Bases de datos, colecciones y documentos

3. De lo relacional a lo documental

- 3.1. Tablas vs. Colecciones
- 3.2. Filas vs. Documentos
- 3.3. Tipos de datos: comparativa
- 3.4. Normalización vs. Denormalización

4. CRUD en MongoDB comparado con SQL

- 4.1. SELECT → find()
- 4.2. INSERT → insertOne / insertMany
- 4.3. UPDATE → updateOne / updateMany
- 4.4. DELETE → deleteOne / deleteMany
- 4.5. Operadores comunes: WHERE, LIKE, IN, BETWEEN

5. Índices y rendimiento

- 5.1. Tipos de índices en MongoDB
- 5.2. Comparación con índices de SQL Server
- 5.3. Análisis de consultas y explain()

6. Agregaciones y consultas avanzadas

- 6.1. Aggregation Pipeline vs. GROUP BY
- 6.2. Proyecciones, filtros y ordenamientos
- 6.3. \$lookup vs. JOIN
- 6.4. Transformaciones y estadísticas



7. Seguridad y control de acceso

- 7.1. Autenticación y roles
- 7.2. Auditoría y cifrado
- 7.3. Comparativa con el modelo de seguridad de SQL Server

8. Replicación y alta disponibilidad

- 8.1. Replica Sets
- 8.2. Sharding: particionado horizontal
- 8.3. Comparación con Always On y particiones en SQL Server

9. Copias de seguridad y recuperación

- 9.1. Backups con mongodump/mongorestore
- 9.2. Snapshots y backups programados
- 9.3. Comparativa con SQL Server Agent y tareas de mantenimiento

10. Integración y migración

- 10.1. Herramientas para migrar de SQL Server a MongoDB
- 10.2. ETL y conectividad con otras plataformas
- 10.3. Casos de uso mixtos (SQL + NoSQL)

11. Casos prácticos

- 11.1. Sistema de tickets
- 11.2. Catálogo de productos
- 11.3. Aplicación de seguimiento de pedidos

12. Buenas prácticas y patrones de diseño

- 12.1. Modelado óptimo en MongoDB
- 12.2. Escalabilidad y mantenimiento
- 12.3. Anti-patrones comunes

Apéndices

- A. Glosario SQL → MongoDB
- B. Recursos recomendados
- C. Scripts de ejemplo
- D. Referencias oficiales



Capítulo 1. Introducción

1.1. Objetivo de la guía

Esta guía está dirigida a profesionales de la informática con experiencia previa en **SQL Server**, que desean aprender **MongoDB**, una base de datos NoSQL orientada a documentos. El objetivo principal es facilitar la transición conceptual y práctica entre ambos modelos, mostrando las equivalencias, diferencias y ventajas de cada enfoque.

A través de ejemplos claros y casos reales, aprenderás cómo modelar, consultar y administrar datos en MongoDB, usando como referencia tu conocimiento de SQL Server.

1.2. ¿Por qué MongoDB?

MongoDB ha ganado popularidad por su flexibilidad, escalabilidad horizontal y enfoque basado en documentos. Algunas razones para considerar MongoDB en proyectos nuevos o existentes:

- **No requiere esquemas rígidos**, lo que permite una evolución ágil del modelo de datos.
- **Escala fácilmente en horizontal** con sharding, ideal para grandes volúmenes de datos.
- **Desempeño optimizado** para operaciones de lectura y escritura a gran escala.
- Amplio ecosistema de herramientas y soporte para múltiples lenguajes.

En entornos donde los datos cambian frecuentemente, o cuando se requiere alta disponibilidad y flexibilidad, MongoDB resulta una opción poderosa.

1.3. Diferencias clave entre SQL Server y MongoDB

Concepto	SQL Server	MongoDB
Tipo de base de datos	Relacional	No relacional (documental)
Estructura	Tablas con filas y columnas	Colecciones con documentos JSON
Esquema	Fijo, definido con anterioridad	Flexible, dinámico
Lenguaje de consulta	T-SQL	BSON queries y Aggregation Pipeline
Integridad referencial	Claves foráneas y constraints	Manual, usando referencias o embed
Escalabilidad	Vertical principalmente	Horizontal (sharding)
Transacciones	Completo soporte ACID	Soporte ACID a nivel de documento
JOINS	Complejos y potentes	Limitados (\$lookup)



En **MongoDB**, el **sharding** es el mecanismo que permite **escalar horizontalmente** una base de datos dividiendo sus datos en múltiples fragmentos llamados **shards**. Cada shard es una instancia independiente de MongoDB que almacena solo una parte del conjunto total de datos.

🔧 ¿Cómo funciona el sharding en MongoDB?

El sistema de sharding en MongoDB está compuesto por tres componentes principales:

1. **Shard**: almacena una porción de los datos.
2. **Mongos**: es el router. Recibe las consultas del cliente y las dirige al shard correspondiente.
3. **Config Servers**: guardan el metadato de los shards (por ejemplo, qué shard tiene qué rango de datos).

■ Proceso de Sharding:

1. Se **elige una colección** que se quiere shardear.
2. Se **define una clave de partición** (shard key), por ejemplo: `user_id`.
3. MongoDB divide los datos en **chunks** según la clave.
4. Los chunks se distribuyen entre los diferentes shards.

✅ Ventajas:

- Permite manejar **grandes volúmenes de datos** que no cabrían en un solo servidor.
- Mejora el **rendimiento y la disponibilidad**.
- Escalabilidad **horizontal** agregando más shards.

⚠️ Desventajas:

- Elegir mal la **shard key** puede provocar desequilibrio (hotspots).
- Consultas complejas pueden verse afectadas si deben consultar varios shards.

MongoDB gestiona automáticamente la mayoría del proceso, pero **diseñar bien la shard key es crucial**.



✓ Ventajas de MongoDB sobre SQL

1. Modelo flexible (sin esquema fijo)

MongoDB usa documentos JSON (BSON), lo que permite almacenar estructuras dinámicas sin necesidad de definir previamente un esquema. Ideal para datos que cambian con el tiempo.

2. Escalabilidad horizontal nativa

Con sharding, MongoDB escala fácilmente añadiendo más servidores, algo que en SQL suele ser más complejo y limitado.

3. Alta disponibilidad automática

ReplicaSet permite tener copias automáticas de los datos para tolerancia a fallos. En SQL esto suele requerir configuraciones más manuales.

4. Velocidad en escritura

MongoDB está optimizado para insertos rápidos y gran volumen de datos, especialmente cuando no se requieren transacciones complejas.

5. Mejor para datos semi-estructurados o no estructurados

MongoDB gestiona bien JSON, listas, mapas y estructuras anidadas que son difíciles de manejar en SQL sin múltiples tablas y relaciones.

6. Desarrollo ágil

La flexibilidad del esquema permite iterar y cambiar el diseño de datos rápidamente, lo que favorece proyectos con evolución rápida.

⊘ Pero cuidado:

MongoDB **no reemplaza totalmente a SQL**. Para datos altamente estructurados, con relaciones complejas, integridad referencial y transacciones estrictas, **SQL es más adecuado**.



Capítulo 2. Fundamentos de MongoDB

2.1. Arquitectura general

MongoDB es una base de datos orientada a documentos que utiliza **documentos BSON** (una representación binaria de JSON) como unidad básica de almacenamiento. Su arquitectura se basa en:

- **mongod**: el proceso principal del servidor de base de datos.
- **mongos**: el router que permite la distribución de datos en clústeres con sharding.
- **Mongo Shell / drivers**: permiten interactuar con la base de datos desde lenguajes como JavaScript, Python, C#, etc.
- **Replica Set**: conjunto de nodos que mantienen copias idénticas para alta disponibilidad.
- **Sharding**: particionado automático de datos en varios servidores.

MongoDB está diseñado para distribuir los datos y cargas de trabajo en varios nodos, favoreciendo la escalabilidad horizontal.

2.2. Instalación y configuración

En Windows:

1. Descarga el instalador desde [mongodb.com](https://www.mongodb.com).
2. Sigue los pasos del asistente y selecciona “Complete”.
3. Habilita la instalación como servicio.
4. Asegúrate de añadir `mongod` y `mongo` al PATH.
5. Inicia el servicio desde el `Services.msc`.

En Linux:

```
sudo apt install -y mongodb
sudo systemctl start mongodb
```

Configuración básica:

- **Directorio de datos**: `C:\data\db` o `/var/lib/mongodb`
- **Archivo de configuración**: `mongod.cfg` o `/etc/mongod.conf`
- Puedes establecer puerto, autenticación, réplica, etc.



2.3. MongoDB Compass y la shell

- **MongoDB Shell (mongosh):** Interfaz de línea de comandos para ejecutar operaciones.

Ejemplo:

```
use miBase
db.miColeccion.insertOne({ nombre: "Oscar", edad: 30 })
```

- **MongoDB Compass:** Interfaz gráfica para explorar bases de datos, ejecutar queries y ver estadísticas. Muy útil para usuarios que vienen de SQL Server Management Studio.

2.4. Bases de datos, colecciones y documentos

MongoDB organiza los datos así:

- **Base de datos (database):** Equivalente a una base en SQL Server.
- **Colección (collection):** Similar a una tabla, pero sin esquema fijo.
- **Documento (document):** Es como una fila, pero en formato JSON/BSON.

Ejemplo de documento:

```
{
  "nombre": "Beatriz",
  "edad": 25,
  "email": "bea@ejemplo.com",
  "direccion": {
    "ciudad": "Palencia",
    "pais": "España"
  }
}
```

⚠ A diferencia de SQL Server, diferentes documentos en una misma colección pueden tener estructuras distintas.



Capítulo 3. De lo relacional a lo documental

3.1. Tablas vs. Colecciones

En SQL Server, los datos se organizan en **tablas** con un esquema fijo. En MongoDB, se utilizan **colecciones**, que agrupan **documentos JSON** sin necesidad de definir previamente las columnas o su tipo.

SQL Server	MongoDB
Tabla	Colección
Fila (row)	Documento
Columna (field)	Clave del documento

💡 MongoDB no necesita `CREATE TABLE`. Puedes insertar directamente en una colección y se crea automáticamente.

3.2. Filas vs. Documentos

Una fila en SQL Server contiene valores atómicos organizados en columnas. En MongoDB, un documento es una estructura jerárquica tipo JSON, capaz de anidar objetos y arrays.

Ejemplo en SQL Server:

```
INSERT INTO Clientes (Nombre, Edad) VALUES ('Luis', 40);
```

Equivalente en MongoDB:

```
db.Clientes.insertOne({ nombre: "Luis", edad: 40 })
```

🔗 MongoDB permite guardar subdocumentos y arrays sin normalizar, algo no permitido directamente en SQL Server.

3.3. Tipos de datos: comparativa

SQL Server	MongoDB (BSON)
INT	Int32, Int64
VARCHAR / NVARCHAR	String
DATETIME	Date
BIT	Boolean



SQL Server	MongoDB (BSON)
FLOAT	Double
BINARY	BinData
UNIQUEIDENTIFIER	ObjectId

 **ObjectId** es el tipo de ID por defecto en MongoDB. Es único, con marca temporal y evita colisiones.

3.4. Normalización vs. Denormalización

En SQL Server, se recomienda normalizar las tablas para eliminar redundancias, usando claves foráneas.

MongoDB, en cambio, favorece la **denormalización**, embebiendo documentos dentro de otros cuando el acceso conjunto es frecuente.

Ejemplo:

SQL Server:

- Tabla Pedidos
- Tabla Clientes
- Relación por ClienteID

MongoDB:

```
{
  "pedidoID": 123,
  "cliente": {
    "nombre": "Sofía",
    "email": "sofia@ejemplo.com"
  },
  "items": [
    { "producto": "Camisa", "cantidad": 2 },
    { "producto": "Pantalón", "cantidad": 1 }
  ]
}
```

✦ Denormalizar mejora el rendimiento en lecturas frecuentes, pero requiere más cuidado al actualizar datos repetidos.



Capítulo 4. CRUD en MongoDB comparado con SQL

4.1. SELECT → find()

SQL Server:

```
SELECT * FROM Clientes WHERE Edad > 30;
```

MongoDB:

```
db.Clientes.find({ edad: { $gt: 30 } })
```

- find() es la función básica de lectura.
- Los operadores (\$gt, \$lt, \$eq, etc.) sustituyen a los operadores SQL.

4.2. INSERT → insertOne() / insertMany()

SQL Server:

```
INSERT INTO Productos (Nombre, Precio) VALUES ('Teclado', 29.99);
```

MongoDB:

```
db.Productos.insertOne({ nombre: "Teclado", precio: 29.99 })
```

Para múltiples registros:

```
db.Productos.insertMany([  
  { nombre: "Ratón", precio: 15.5 },  
  { nombre: "Monitor", precio: 199.99 }  
])
```

👉 No es necesario definir el esquema previamente.

4.3. UPDATE → updateOne() / updateMany()

SQL Server:

```
UPDATE Clientes SET Edad = 35 WHERE Nombre = 'Carlos';
```

MongoDB:

```
db.Clientes.updateOne(  
  { nombre: "Carlos", edad: 30 },  
  { $set: { edad: 35 } })
```




```
{ nombre: "Carlos" },
{ $set: { edad: 35 } }
}
```

- \$set modifica solo los campos indicados.
- También existen \$inc, \$unset, \$push, etc.

⚠ Si no se encuentra el documento, no se actualiza nada (salvo que se use `upsert: true`).

4.4. DELETE → `deleteOne()` / `deleteMany()`

SQL Server:

```
DELETE FROM Productos WHERE Precio < 10;
```

MongoDB:

```
db.Productos.deleteMany({ precio: { $lt: 10 } })
```

✅ MongoDB diferencia entre `deleteOne` y `deleteMany`, como TOP 1 y múltiples filas en SQL.

4.5. Operadores comunes: WHERE, LIKE, IN, BETWEEN

SQL Server	MongoDB equivalente
WHERE edad > 30	{ edad: { \$gt: 30 } }
LIKE '%ana%'	{ nombre: /ana/i }
IN ('Madrid', 'Sevilla')	{ ciudad: { \$in: ['Madrid', 'Sevilla'] } }
BETWEEN 10 AND 20	{ precio: { \$gte: 10, \$lte: 20 } }

💡 MongoDB usa expresiones regulares para búsquedas tipo LIKE.



Capítulo 5. Índices y rendimiento

5.1. Tipos de índices en MongoDB

MongoDB permite crear diversos tipos de índices para acelerar consultas:

- **Índice simple:** sobre un solo campo.
- `db.Clientes.createIndex({ nombre: 1 })` // 1 = ascendente
- **Índice compuesto:** combina varios campos.
- `db.Pedidos.createIndex({ clienteId: 1, fecha: -1 })`
- **Índice único:** evita duplicados.
- `db.Usuarios.createIndex({ email: 1 }, { unique: true })`
- **Índice geoespacial, texto, hash, wildcard, etc.**

🔍 Usa `getIndexes()` para listar los índices de una colección.

5.2. Comparación con índices de SQL Server

Característica	SQL Server		MongoDB
Índice simple	Sí	Sí	
Índice compuesto	Sí	Sí	
Índice único	Sí (UNIQUE)	Sí ({ unique: true })	
Índices filtrados	Sí (WHERE)	No directamente	
Índices de texto	Limitado	Nativo (text)	
Reorganización de índices Recomendado MongoDB lo maneja internamente			

✦ MongoDB no tiene índices **clustered** como SQL Server; todos son **secundarios**.



5.3. Análisis de consultas y `explain()`

MongoDB ofrece `explain()` para estudiar el plan de ejecución:

```
db.Clientes.find({ edad: { $gt: 30 } }).explain("executionStats")
```

- Muestra si se usó índice (`IXSCAN`) o no (`COLLSCAN`).
- También revela número de documentos escaneados y devueltos.

Métricas importantes:

- `executionTimeMillis`
- `totalDocsExamined`
- `totalKeysExamined`

💡 Similar a `SET STATISTICS IO` y `EXPLAIN` en SQL Server.



Capítulo 6. Agregaciones y consultas avanzadas

6.1. Aggregation Pipeline vs. GROUP BY

En SQL Server usamos `GROUP BY` para agrupar y resumir datos.

SQL Server:

```
SELECT ciudad, COUNT(*) FROM Clientes GROUP BY ciudad;
```

MongoDB:

```
db.Clientes.aggregate([
  { $group: { _id: "$ciudad", total: { $sum: 1 } } }
])
```

MongoDB utiliza **Aggregation Pipeline**, una secuencia de etapas (`$match`, `$group`, `$sort`, etc.) que procesan los documentos.

6.2. Proyecciones, filtros y ordenamientos

- **\$project**: selecciona y transforma campos (equivalente a `SELECT campo1, campo2`).
- **\$match**: filtra documentos (similar a `WHERE`).
- **\$sort**: ordena los resultados.

Ejemplo:

```
db.Productos.aggregate([
  { $match: { categoria: "Electrónica" } },
  { $project: { nombre: 1, precio: 1, _id: 0 } },
  { $sort: { precio: -1 } }
])
```

🧠 Similar a:

```
SELECT nombre, precio FROM Productos
WHERE categoria = 'Electrónica'
ORDER BY precio DESC;
```



6.3. \$lookup vs. JOIN

MongoDB no tiene JOINS tradicionales, pero permite uniones limitadas usando \$lookup.

SQL Server:

```
SELECT C.Nombre, P.Producto
FROM Clientes C
JOIN Pedidos P ON C.ClienteID = P.ClienteID;
```

MongoDB:

```
db.Clientes.aggregate([
  {
    $lookup: {
      from: "Pedidos",
      localField: "clienteId",
      foreignField: "clienteId",
      as: "pedidos"
    }
  }
])
```

⚠ \$lookup está limitado a uniones entre colecciones del mismo servidor.

6.4. Transformaciones y estadísticas

Puedes usar otras operaciones como:

- \$sum, \$avg, \$min, \$max: funciones de agregación.
- \$addFields, \$set: para añadir campos.
- \$unwind: para descomponer arrays.
- \$facet: para crear múltiples pipelines en paralelo.
- \$bucket: para crear agrupaciones por rango (como CASE WHEN).

Ejemplo:

```
db.Pedidos.aggregate([
  { $group: { _id: "$cliente", total: { $sum: "$importe" } } },
  { $sort: { total: -1 } }
])
```

🔗 MongoDB puede realizar análisis complejos sin necesidad de procedimientos almacenados.



Capítulo 7. Seguridad y control de acceso

7.1. Autenticación y roles

MongoDB implementa un sistema robusto de autenticación basado en usuarios y roles.

- Para habilitar autenticación, agrega en `mongod.conf`:
 - `security:`
 - `authorization: enabled`
- Creación de usuario administrador:
 - `use admin`
 - `db.createUser({`
 - `user: "admin",`
 - `pwd: "passwordSegura",`
 - `roles: [ { role: "userAdminAnyDatabase", db: "admin" } ]`
 - `})`
- Autenticación:
 - `mongosh -u admin -p --authenticationDatabase admin`

Roles comunes:

Rol MongoDB Equivalente en SQL Server

read / readWrite db_datareader / db_datawriter

dbAdmin db_owner

userAdmin securityadmin

root sysadmin

7.2. Auditoría y cifrado

Auditoría:

- MongoDB Enterprise ofrece un sistema de auditoría integrado.
- En Community Edition, puedes registrar acciones desde el sistema o mediante logs de acceso.

Cifrado:

- **En tránsito:** Usando TLS/SSL (como SQL Server con certificados).
- **En reposo:** MongoDB Enterprise incluye cifrado nativo del disco.

🔒 Es recomendable usar conexiones seguras y restringir el acceso a puertos desde la red.



7.3. Comparativa con el modelo de seguridad de SQL Server

Característica	SQL Server	MongoDB
Autenticación integrada	Active Directory, SQL Auth	MongoDB Auth, LDAP (Enterprise)
Control por roles	Sí	Sí
Cifrado en tránsito	TLS	TLS
Cifrado en reposo	TDE	Enterprise Edition
Auditoría	SQL Audit	Solo en MongoDB Enterprise

⚠ MongoDB Community tiene limitaciones en auditoría y cifrado avanzado respecto a la versión Enterprise.



Capítulo 8. Replicación y alta disponibilidad

8.1. Replica Sets

Un **Replica Set** es un grupo de servidores MongoDB que mantienen copias sincronizadas de los datos, proporcionando **alta disponibilidad**.

Componentes:

- **Primary:** nodo principal que acepta escrituras.
- **Secondary:** nodos que replican los datos del primario.
- **Arbiter (opcional):** participa en elecciones pero no almacena datos.

Comandos básicos:

```
mongod --replSet "rs0"  
rs.initiate()  
rs.add("servidor2:27017")  
rs.add("servidor3:27017")
```

 Si el primario falla, los secundarios votan para elegir un nuevo primario.

8.2. Sharding: particionado horizontal

MongoDB permite escalar horizontalmente distribuyendo datos entre múltiples nodos (shards).

Componentes:

- **Shard:** contiene una porción de los datos.
- **Config Servers:** guardan metadatos.
- **mongos:** router que dirige las consultas.

Ejemplo:

```
mongos --configdb configReplSet/host1,host2,host3
```

 Sharding se recomienda para colecciones grandes (millones de documentos) con alta carga.

8.3. Comparación con Always On y particiones en SQL Server



Característica	SQL Server	MongoDB
Alta disponibilidad	Always On, Clustering	Replica Sets
Particionado de datos	Particiones de tabla	Sharding
Elección automática	Solo en Clustering	Automática en Replica Sets
Redirección de consultas	Manual (failover partner)	Transparente vía <code>mongos</code>

🔧 MongoDB está diseñado desde el principio para distribuir datos, mientras que SQL Server tradicionalmente escala verticalmente.



Capítulo 9. Copias de seguridad y recuperación

9.1. Backups con `mongodump` / `mongorestore`

MongoDB incluye utilidades de línea de comandos para exportar e importar datos fácilmente.

Copia de seguridad (`mongodump`):

```
mongodump --db miBase --out /copias/backup/
```

- Exporta los datos en formato BSON.
- Puedes respaldar toda la base o solo una colección.

Restauración (`mongorestore`):

```
mongorestore --db miBase /copias/backup/miBase
```

💡 Ideal para copias rápidas o migraciones.

9.2. Snapshots y backups programados

En entornos de producción:

- Es común usar **snapshots del sistema de archivos** si se usa almacenamiento persistente.
- MongoDB Atlas (nube) permite backups automáticos y restauraciones con un clic.
- También se pueden programar tareas con `cron` (Linux) o el Programador de tareas (Windows).

Ejemplo en Linux:

```
0 2 * * * mongodump --db miBase --out /var/backups/mibase_$(date +%F)
```

9.3. Comparativa con SQL Server Agent y tareas de mantenimiento

Función	SQL Server	MongoDB
Backups programados	SQL Server Agent / Maintenance Plans	<code>cron</code> , scripts, MongoDB Atlas
Backups incrementales	Sí (diferenciales / logs)	Solo en Atlas o herramientas externas
Backup en caliente	Sí	Solo con Filesystem Snapshots o Atlas



Función		SQL Server	MongoDB
Herramientas GUI	SSMS		Compass (limitado) / Atlas

🔒 Recuerda proteger las copias: cifrado, permisos y almacenamiento seguro.



Capítulo 10. Integración y migración

10.1. Herramientas para migrar de SQL Server a MongoDB

Existen varias herramientas que ayudan a migrar datos desde SQL Server a MongoDB:

- **MongoDB Connector for BI:** permite usar SQL para consultar datos en MongoDB.
- **MongoDB Compass (Import Wizard):** permite importar desde CSV o JSON.
- **Pentaho, Talend, SSIS (ETL):** permiten flujos de transformación de datos entre SQL Server y MongoDB.
- **mongoimport + bcp o sqlcmd:** exportar desde SQL y luego importar.

Ejemplo:

1. Exportar desde SQL Server:

```
bcp "SELECT * FROM Clientes" queryout clientes.csv -c -t, -S . -U usuario -P password
```

2. Importar en MongoDB:

```
mongoimport --db miBase --collection Clientes --type csv --file clientes.csv --headerline
```

10.2. ETL y conectividad con otras plataformas

MongoDB se puede integrar con:

- **Lenguajes de programación:** Python, Node.js, C#, Java, etc.
- **ETL Tools:** Talend, Apache NiFi, Informatica.
- **Power BI:** vía ODBC o conectores intermedios.
- **Apache Kafka, Spark, RabbitMQ:** para flujos en tiempo real.

✚ MongoDB funciona bien como repositorio NoSQL en arquitecturas híbridas.



10.3. Casos de uso mixtos (SQL + NoSQL)

En muchas empresas, MongoDB **no sustituye completamente a SQL Server**, sino que **lo complementa**:

SQL Server	MongoDB
Datos financieros y estructurados	Datos semiestructurados o flexibles
Informes y reporting	Aplicaciones web/móvil en tiempo real
Integridad referencial estricta	Agilidad y escalabilidad

Ejemplo real:

- SQL Server gestiona inventario y contabilidad.
- MongoDB guarda el historial de navegación y preferencias del usuario.

 Conectores ETL pueden mantener sincronizados ambos mundos.



Capítulo 11. Casos prácticos

11.1. Sistema de tickets

Escenario:

Aplicación para gestionar incidencias de soporte técnico.

Modelo SQL tradicional:

- Tabla Tickets: ID, Título, Descripción, Estado, Fecha, UsuarioID
- Tabla Usuarios: ID, Nombre, Email

MongoDB:

```
{
  "_id": ObjectId("..."),
  "titulo": "Error al iniciar sesión",
  "descripcion": "No se puede acceder al sistema desde Chrome",
  "estado": "Abierto",
  "fecha": ISODate("2025-06-26T09:30:00Z"),
  "usuario": {
    "nombre": "Laura",
    "email": "laura@empresa.com"
  }
}
```

✅ Se embebe el usuario, ya que no cambiará frecuentemente y simplifica la lectura.

11.2. Catálogo de productos

Escenario:

Una tienda online necesita mostrar productos, categorías y variantes.

SQL Server:

- Tabla Productos
- Tabla Categorías
- Tabla Variantes



MongoDB:

```
{
  "nombre": "Zapatillas Running",
  "precio": 89.99,
  "categoria": "Deportes",
  "variantes": [
    { "talla": 40, "stock": 12 },
    { "talla": 41, "stock": 5 }
  ]
}
```

 MongoDB permite almacenar arrays de variantes directamente dentro del producto.

11.3. Aplicación de seguimiento de pedidos

Escenario:

Registrar los cambios de estado de un pedido a lo largo del tiempo.

MongoDB:

```
{
  "pedidoId": "P12345",
  "cliente": "Carlos Sánchez",
  "fecha": ISODate("2025-06-25T15:20:00Z"),
  "eventos": [
    { "estado": "Procesado", "hora": "15:21" },
    { "estado": "Enviado", "hora": "16:00" },
    { "estado": "Entregado", "hora": "18:10" }
  ]
}
```

 Ideal para representar flujos de eventos o auditorías sin necesidad de tablas secundarias.



Capítulo 12. Buenas prácticas y patrones de diseño

12.1. Modelado óptimo en MongoDB

MongoDB permite varias formas de estructurar datos. Para elegir bien:

- **Embebe (subdocumentos) si:**
 - Los datos se consultan siempre juntos.
 - No crecen ilimitadamente.
- **Referencia (como SQL) si:**
 - El subdocumento se reutiliza en muchos lugares.
 - Es necesario mantenerlo actualizado por separado.

Ejemplo:

- **Embebido:** dirección dentro del usuario.
- **Referenciado:** autor referenciado desde varios libros.

💡 MongoDB es flexible, pero un mal modelo puede afectar el rendimiento.

12.2. Escalabilidad y mantenimiento

Recomendaciones clave:

- **Índices adecuados:** evita consultas lentas.
- **Evita documentos demasiado grandes** (>16MB).
- **Haz backups regularmente.**
- **Replica tus datos para alta disponibilidad.**
- **Usa sharding si el volumen lo justifica.**

⚙️ Evalúa el acceso a los datos: consultas, escritura, frecuencia y carga.



12.3. Anti-patrones comunes

Anti-patrón

Abusar de referencias

Documentos gigantes (logs infinitos)

No usar índices

Imitar el modelo relacional

No validar los datos

Por qué evitarlo

Aumenta la complejidad y ralentiza lecturas

Superan límites de tamaño y degradan el rendimiento

Las consultas hacen `COLLSCAN`

Pierdes flexibilidad y velocidad

Riesgo de inconsistencia

❌ MongoDB no fuerza esquemas, pero eso no significa dejar los datos sin control.



Apéndices

Apéndice A. Glosario SQL → MongoDB

Concepto en SQL Server	Equivalente en MongoDB
Tabla	Colección
Fila (Row)	Documento
Columna	Clave (campo)
Primary Key	<code>_id</code>
FOREIGN KEY	Referencia manual / <code>\$lookup</code>
SELECT	<code>find()</code>
INSERT	<code>insertOne()</code> / <code>insertMany()</code>
UPDATE	<code>updateOne()</code> / <code>updateMany()</code>
DELETE	<code>deleteOne()</code> / <code>deleteMany()</code>
WHERE	Filtro con operadores (<code>\$eq</code> , <code>\$gt</code>)
JOIN	<code>\$lookup</code>
GROUP BY	<code>\$group</code>
ORDER BY	<code>\$sort</code>
Índice (<code>CREATE INDEX</code>)	<code>createIndex()</code>
Transacción (<code>BEGIN...</code>)	<code>session.startTransaction()</code>
Stored Procedure	Código en aplicación / Aggregation

Apéndice B. Recursos recomendados

-  [Documentación oficial de MongoDB](#)
-  [MongoDB Atlas](#): MongoDB como servicio.
-  Libro: *MongoDB: The Definitive Guide* – O'Reilly.
-  Blog: <https://www.practical-mongodb.com>
-  Cursos online: Udemy, Coursera, freeCodeCamp, Platzi.
-  Herramienta de migración: [Studio 3T SQL → MongoDB](#)



Apéndice C. Scripts de ejemplo

Crear índice:

```
db.Usuarios.createIndex({ email: 1 }, { unique: true })
```

Agrupación por campo:

```
db.Clientes.aggregate([  
  { $group: { _id: "$ciudad", total: { $sum: 1 } } }  
])
```

Inserción masiva:

```
db.Productos.insertMany([  
  { nombre: "Tablet", precio: 150 },  
  { nombre: "Smartphone", precio: 299 }  
])
```

Consulta con \$in:

```
db.Usuarios.find({ ciudad: { $in: ["Madrid", "Barcelona"] } })
```

Apéndice D. Referencias oficiales

- <https://www.mongodb.com/docs/manual/>
 - <https://www.mongodb.com/docs/drivers/>
 - <https://api.mongodb.com/>
 - <https://www.mongodb.com/developer/>
 - <https://www.mongodb.com/docs/atlas/>
-



Agradecimientos

Gracias a todas las personas que hacen posible el aprendizaje abierto y práctico: a quienes comparten ejemplos, resuelven dudas en foros, documentan diferencias entre tecnologías, y crean puentes entre mundos que parecían opuestos.

A la comunidad de desarrolladores, DBA y profesionales que siguen creyendo en el poder de aprender algo nuevo, incluso cuando ya dominan otro enfoque. Y a ti, lector o lectora, por confiar en esta guía para abrirte camino en el universo NoSQL desde tu sólida experiencia en bases de datos relacionales.

Cambiar no es renunciar, es evolucionar.
¡Nos vemos en el siguiente reto tecnológico!

